

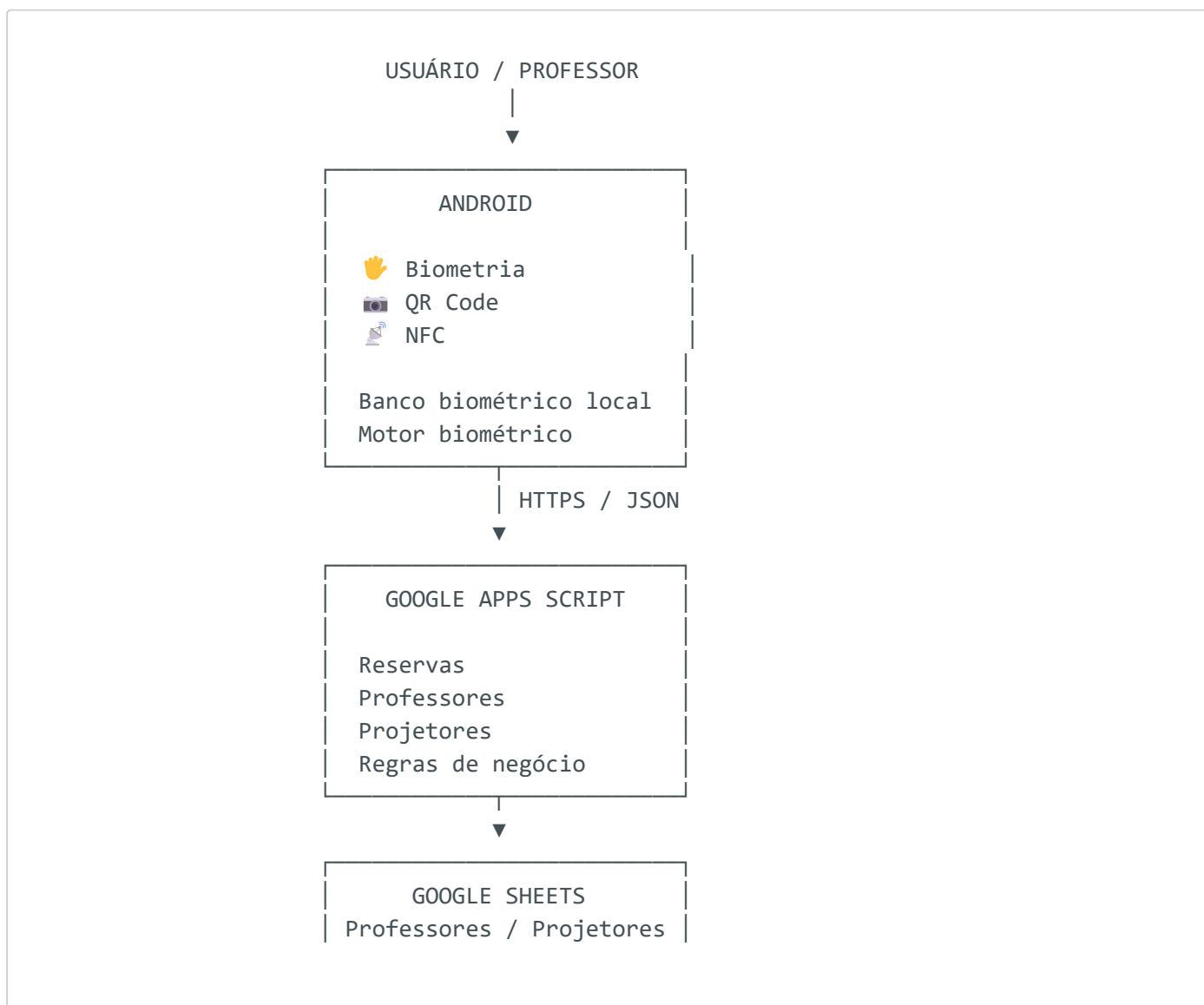
Arquitetura da Solução — Reserva de Projetores CCT

1. Visão geral

A solução utiliza um terminal Android com três formas de identificação: impressão digital, QR Code e NFC. A aplicação de negócio será desenvolvida com Google Apps Script, aproveitando Google Sheets/Drive/Docs.

A principal premissa é a **independência de fabricante**: o Hamster DX é apenas um possível leitor biométrico.

2. Arquitetura geral

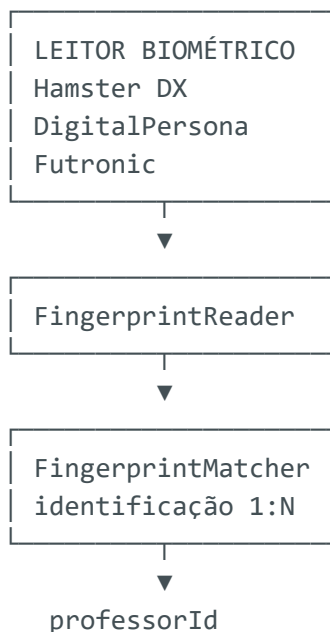


3. Objetivos

- Identificar o professor por impressão digital.
- Identificar o projetor por QR Code ou NFC.
- Permitir reservas de projetores.
- Usar Android como terminal multimodal.
- Manter templates biométricos no próprio Android.
- Não enviar imagens/templates biométricos ao Google Apps Script.
- Usar Google Apps Script como camada de aplicação.
- Usar Google Sheets como armazenamento inicial das reservas e dados administrativos.
- Permitir troca futura do leitor sem reescrever a aplicação.

4. Princípio de independência

Separar leitor, captura, motor biométrico e aplicação:



Interface conceitual:

```
public interface FingerprintReader {  
    byte[] capture();  
    boolean isConnected();  
}
```

```
public interface FingerprintMatcher {  
    FingerprintTemplate createTemplate(byte[] image);  
    IdentificationResult identify(FingerprintTemplate template);  
}
```

5. Android

O Android será o **Terminal Biométrico Multimodal CCT**.

Responsabilidades:

- Capturar impressão digital.
- Executar identificação biométrica localmente.
- Ler QR Code pela câmera.
- Ler NFC pelo hardware nativo.
- Armazenar templates biométricos localmente.
- Comunicar-se com o Google Apps Script.
- Exibir identificação e confirmação de reserva.

Estrutura sugerida:

```
br.unifor.cct  
├── biometria  
│   ├── FingerprintReader.java  
│   ├── FingerprintMatcher.java  
│   ├── BiometriaService.java  
│   ├── HamsterDxReader.java  
│   └── ResultadoBiometria.java  
├── qrcode  
│   ├── QrCodeService.java  
│   └── ResultadoQrCode.java  
├── nfc  
│   ├── NfcService.java  
│   └── ResultadoNfc.java  
├── storage  
│   ├── BiometricDatabase.java  
│   └── ProfessorRepository.java  
├── api  
└── AppsScriptClient.java
```

```

├── ReservaApi.java
├── model
│   ├── Professor.java
│   ├── Projetoor.java
│   └── Reserva.java
└── ui
    ├── MainActivity.java
    ├── IdentificacaoActivity.java
    └── ReservaActivity.java

```

6. Conexão do leitor

Para leitores USB, como o Hamster DX:

```

Celular Android
  | USB-C
  ▼
Adaptador USB-C OTG
  | USB-A
  ▼
Leitor biométrico USB

```

O adaptador OTG resolve a conexão física; a compatibilidade também depende do suporte USB Host/OTG do aparelho e do SDK Android do leitor.

7. Biometria local

```

Professor coloca o dedo
  ▼
Leitor biométrico
  ▼
Android
  ▼
Captura
  ▼
Template
  ▼
Comparação 1:N
  ▼
Professor identificado
  ▼
professorId

```

O Google Apps Script recebe apenas o identificador do professor.

Exemplo:

```
{  
  "professorId": 123  
}
```

8. Armazenamento biométrico

Os templates biométricos ficam no Android.

```
Professor  
├── id  
├── nome  
├── matrícula  
├── template_biometrico  
├── algoritmo  
├── formato  
├── versão  
└── ativo
```

Não utilizar imagem bruta da digital como cadastro principal. Preferir template biométrico e proteger o armazenamento com os mecanismos de segurança do Android, incluindo armazenamento privado e Android Keystore quando aplicável.

9. QR Code

O QR Code identifica o projetor.

```
QR Code → PAT-001 → Android → Apps Script
```

Exemplo:

```
{  
  "tipo": "PROJETOR",  
  "codigo": "PAT-001"  
}
```

10. NFC

Tag NFC → UID/identificador → Android → Apps Script

Exemplo:

```
{  
  "tipo": "PROJETOR",  
  "codigo": "NFC-PAT-001"  
}
```

11. Processo de reserva

1. Professor coloca a digital
2. Android identifica professor
3. Professor lê QR Code ou NFC do projetor
4. Android identifica projetor
5. Android consulta Apps Script
6. Apps Script verifica disponibilidade
7. Reserva é registrada no Google Sheets
8. Android apresenta confirmação

12. Google Apps Script

O Apps Script será a camada de aplicação e deverá:

- receber requisições do Android;
- validar professor e projetor;
- verificar disponibilidade;
- criar/cancelar reservas;
- consultar reservas;
- manter histórico;
- integrar com Google Sheets;
- integrar com Drive/Docs quando necessário.

Exemplos de operações:

```
POST /identificar
POST /projedor
POST /reserva
GET  /reserva
DELETE /reserva
```

No Web App do Apps Script, essas operações podem ser implementadas como ações/parâmetros de uma mesma URL `doGet/doPost`.

13. Google Sheets

Aba **PROFESSORES**

ID	Matrícula	Nome	Ativo
1	12345	Professor A	SIM
2	12346	Professor B	SIM

Aba **PROJETORES**

ID	Patrimônio	Local	Ativo
1	PAT-001	CCT-101	SIM
2	PAT-002	CCT-102	SIM

Aba **RESERVAS**

ID	Professor	Projedor	Data	Início	Fim	Status
1	1	PAT-001	22/09/2026	19:00	21:00	ATIVA

Aba **CONFIGURACAO**

Parâmetros como horários permitidos, antecedência, limite de reservas e regras administrativas.

14. Comunicação Android → Apps Script

O Android não envia dados biométricos.

Exemplo:

```
{
  "acao": "CRIAR_RESERVA",
  "professorId": 123,
  "projetoId": "PAT-001",
  "data": "2026-09-22",
  "inicio": "19:00",
  "fim": "21:00"
}
```

Resposta:

```
{
  "sucesso": true,
  "reservaId": 987,
  "mensagem": "Reserva realizada com sucesso."
}
```

15. Segurança e LGPD

Impressão digital é dado pessoal sensível. A arquitetura deve minimizar sua circulação.

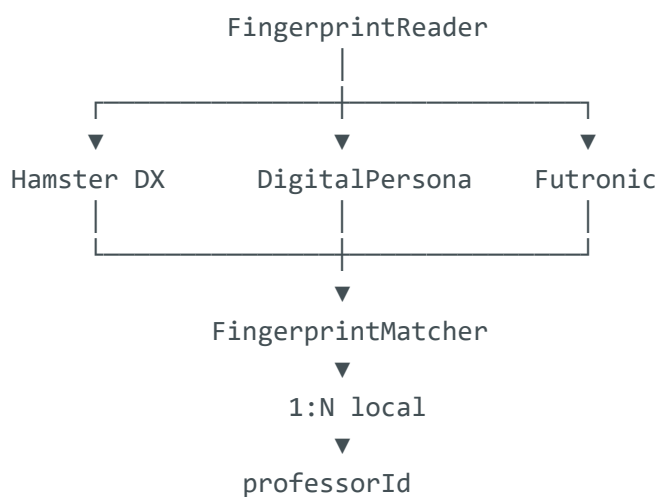
- Não enviar imagem da digital ao Apps Script.
- Não armazenar imagem bruta sem necessidade.
- Manter templates no Android.
- Proteger o banco local.
- Utilizar HTTPS.
- Autenticar o terminal.
- Controlar cadastro/remoção de biometria.
- Registrar operações administrativas.
- Não colocar templates biométricos no Google Sheets.
- Definir retenção e exclusão.

- Aplicar controles compatíveis com LGPD e regras institucionais.

16. Independência tecnológica

A solução deve evitar dependência de:

- Hamster DX;
- um único fabricante de leitor;
- um único motor biométrico;
- Google Sheets como banco definitivo;
- um único modelo de celular.



17. Evolução futura

V1

Android — Apps Script — Google Sheets

V2

Android — API — Spring Boot — Banco

A troca de backend não precisa alterar a lógica do terminal se as interfaces forem mantidas.

18. Responsabilidade dos componentes

Componente	Responsabilidade
Android	Terminal multimodal
Leitor biométrico	Captura
Motor biométrico	Template e identificação 1:N
Banco Android	Templates biométricos
Câmera	QR Code
NFC Android	Tags NFC
Apps Script	Regras de negócio
Google Sheets	Dados administrativos e reservas
Google Drive/Docs	Documentos/relatórios

19. Roadmap

Fase 1 — Android

- ☐ Criar projeto Android em Java.
- ☐ Testar USB Host/OTG.
- ☐ Conectar Hamster DX.
- ☐ Obter SDK Android do leitor.
- ☐ Capturar digital.
- ☐ Criar template.
- ☐ Armazenar template localmente.
- ☐ Implementar 1:N.

Fase 2 — QR Code

- ☐ Implementar leitura pela câmera.

- ☐ Definir padrão do QR Code.
- ☐ Associar QR ao patrimônio.

Fase 3 — NFC

- ☐ Implementar leitura NFC.
- ☐ Definir padrão das tags.
- ☐ Associar NFC ao projetor.

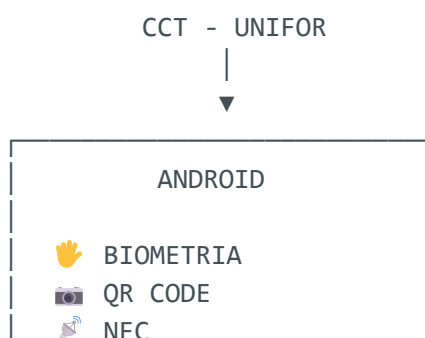
Fase 4 — Apps Script

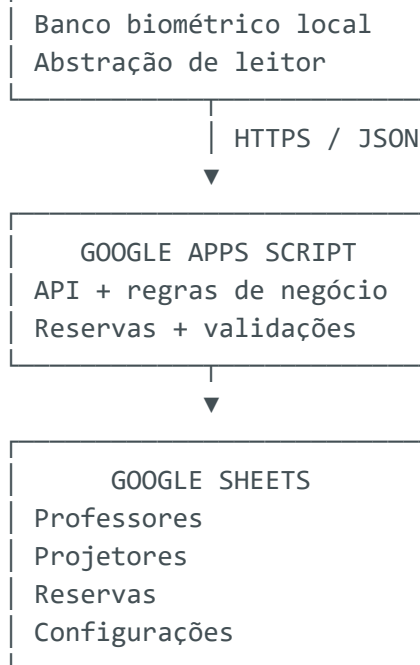
- ☐ Criar Google Sheets.
- ☐ Criar Web App.
- ☐ Implementar professores.
- ☐ Implementar projetores.
- ☐ Implementar reservas.
- ☐ Implementar cancelamento.
- ☐ Implementar histórico.

Fase 5 — Integração

- ☐ Android → Apps Script.
- ☐ professorId → reserva.
- ☐ QR/NFC → projetorId.
- ☐ Criar reserva.
- ☐ Confirmar operação no terminal.

20. Arquitetura final





Conclusão

A solução mantém a biometria **local no Android**, utiliza **Google Apps Script** como aplicação de negócio e **Google Sheets** como armazenamento inicial.

O ponto central é separar **leitor biométrico**, **motor biométrico** e **aplicação**. O Hamster DX pode ser utilizado no desenvolvimento inicial sem transformar o projeto em uma solução dependente do fabricante.

A arquitetura permite evoluir posteriormente para outro leitor, outro motor biométrico ou outro backend sem reconstruir todo o sistema.